

SESSION 4

EXPERIMENT – 7 - CPU Scheduling Algorithms

Theory

1. CPU Scheduling

CPU scheduling is the process of selecting a process from the ready queue and allocating the CPU to it. The main objective of CPU scheduling is to utilize the CPU efficiently and reduce the waiting and turnaround time of processes.

A **CPU scheduler** selects the next process to be executed based on the scheduling algorithm being used.

The important scheduling criteria are:

- **CPU Utilization:** Percentage of time the CPU is busy.
- **Throughput:** Number of processes completed per unit of time.
- **Turnaround Time:** Total time taken from process arrival to completion.
- **Waiting Time:** Time a process spends waiting in the ready queue.
- **Response Time:** Time between submitting a request and receiving the first response.

The basic formulas are:

Turnaround Time:

Turnaround Time = Completion Time - Arrival Time

Waiting Time:

Waiting Time = Turnaround Time - Burst Time

2. Context Switching

A **context switch** occurs when the CPU changes from executing one process to another.

The operating system saves the current process's state, including information such as CPU registers and program counter, and loads the saved state of another process.

Context switching allows multiple processes to share the CPU, but it also introduces **overhead** because the CPU spends some time switching between processes instead of executing processes.

3. Round Robin Scheduling

Round Robin (RR) is a preemptive CPU scheduling algorithm mainly designed for time-sharing systems.

Each process is given a fixed amount of CPU time called a **time quantum**.

If a process does not finish within its time quantum, it is preempted and placed at the end of the ready queue. The CPU is then given to the next process.

Example:

P1 → P2 → P3 → P1 → P2 → P3

Advantages

- Fair to all processes.
- Suitable for interactive and time-sharing systems.
- Prevents one process from occupying the CPU for too long.

Disadvantages

- Very small time quantum causes many context switches.
- Very large time quantum makes it behave similarly to FCFS.

4. Shortest Remaining Time First (SRTF)

Shortest Remaining Time First is the preemptive version of Shortest Job First (SJF).

In SRTF, the process having the **smallest remaining CPU burst time** is selected for execution.

If a new process arrives with a shorter remaining time than the currently running process, the current process is preempted.

Advantages

- Can provide low average waiting time.
- Gives preference to shorter processes.

Disadvantages

- Long processes may experience starvation.
- The scheduler must continuously consider remaining burst times.
- Frequent preemption can cause additional context-switching overhead.

5. Priority Scheduling

In **Priority Scheduling**, every process is assigned a priority. The CPU is allocated to the process having the highest priority.

Priority scheduling can be either:

- **Preemptive:** A newly arrived higher-priority process can interrupt the running process.
- **Non-preemptive:** The running process continues until completion.

A major problem with priority scheduling is **starvation**, where a low-priority process may wait for a very long time.

6. Starvation and Aging

Starvation occurs when a process waits indefinitely because other processes are continuously given preference.

Aging is a technique used to solve starvation. The priority of a waiting process is gradually increased over time.

For example:

Initial priority → 5

After waiting → 4

After more waiting → 3

After more waiting → 2

Eventually, the waiting process receives a sufficiently high priority and gets CPU time.

7. Multilevel Queue Scheduling

In **Multilevel Queue Scheduling**, the ready queue is divided into multiple separate queues based on process characteristics.

For example:

System Processes



Interactive Processes



Batch Processes



Student/Background Processes

Each queue can have its own scheduling algorithm.

For example:

- Foreground queue → Round Robin
- Background queue → FCFS

Processes generally remain in their assigned queue.

8. Multilevel Feedback Queue Scheduling

A **Multilevel Feedback Queue (MLFQ)** is similar to multilevel queue scheduling, but processes can **move between queues** depending on their behavior and CPU requirements.

A process that uses too much CPU time may be moved to a lower-priority queue, while a waiting process may be moved to a higher-priority queue.

This provides flexibility and can help reduce starvation.

9. Race Condition

A **race condition** occurs when multiple processes or threads access shared data at the same time and the final result depends on the order in which they execute.

For example, if two processes simultaneously modify the same variable, the final value may be incorrect.

Race conditions can be prevented using synchronization mechanisms such as:

- Mutexes
- Semaphores
- Monitors
- Locks

10. Critical Section

A **critical section** is the part of a program where a process accesses or modifies shared resources.

Only one process should execute the critical section at a time when the shared resource cannot safely be accessed concurrently.

A correct solution to the critical-section problem should provide:

- **Mutual Exclusion**
- **Progress**
- **Bounded Waiting**

11. Deadlock

A **deadlock** occurs when two or more processes are permanently waiting for resources held by each other.

For example:

Process P1 → holds Resource R1 → waiting for R2

Process P2 → holds Resource R2 → waiting for R1

Neither process can continue.

The four necessary conditions for deadlock are:

1. **Mutual Exclusion**
2. **Hold and Wait**
3. **No Preemption**
4. **Circular Wait**

12. Priority Inversion

Priority inversion occurs when a high-priority process has to wait for a low-priority process because the low-priority process holds a resource required by the high-priority process.

Priority inversion can be reduced using techniques such as **priority inheritance**, where the lower-priority process temporarily inherits the higher priority.

CODE:

C exp6.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7
8      int fd[2];
9      pid_t pid;
10
11      // Create pipe
12      if (pipe(fd) == -1) {
13          perror("pipe");
14          return 1;
15      }
16
17      // Create child process
18      pid = fork();
19
20      if (pid < 0) {
21          perror("fork");
22          return 1;
23      }
24
25      // Child process
26      if (pid == 0) {
27
28          close(fd[1]);
29
30          // Redirect pipe input to standard input
31          dup2(fd[0], STDIN_FILENO);
32
33          close(fd[0]);
34
35          // Execute grep
36          execlp("grep", "grep", "Alice", NULL);
37      }
```

```

37
38     perror("execlp");
39     exit(1);
40 }
41
42 // Parent process
43 else {
44
45     close(fd[0]);
46
47     char data[] =
48         "Alice is studying Operating Systems.\n"
49         "Bob is studying Computer Networks.\n"
50         "Alice completed the pipe experiment.\n"
51         "Charlie is working on DBMS.\n";
52
53     // Write data into pipe
54     write(fd[1], data, sizeof(data));
55
56     close(fd[1]);
57
58     wait(NULL);
59
60     printf("\nParent process completed.\n");
61 }
62
63 return 0;
64 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc exp6.c -o exp6
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./exp6
grep: (standard input): binary file matches

Parent process completed.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ 

```

Result

Thus, **Inter-Process Communication using an anonymous pipe** was successfully implemented. The `pipe()`, `fork()`, and `dup2()` system calls were used to establish

communication between the parent and child processes, and the required data was successfully transferred through the pipe.

EXPERIMENT – 8

Inter-Process Communication Using Anonymous Pipes

Aim

To implement **Inter-Process Communication (IPC)** between a parent process and a child process using an **anonymous pipe** and understand the use of the `pipe()` and `dup2()` system calls.

Description

A pipe provides a **unidirectional communication channel** between processes.

The commonly used system calls are:

- `pipe()` – creates the pipe.
- `fork()` – creates a child process.
- `dup2()` – redirects standard input/output to the pipe.
- `read()` / `write()` – transfers data through the pipe.
- `close()` – closes unused pipe ends.

Code

```
C exp8.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7
8      int fd[2];
9      pid_t pid;
10
11     // Create pipe
12     if (pipe(fd) == -1) {
13         perror("pipe");
14         return 1;
15     }
16
17     // Create child process
18     pid = fork();
19
20     if (pid < 0) {
21         perror("fork");
22         return 1;
23     }
24
25     // Child process
26     if (pid == 0) {
27
28         close(fd[1]);
29
30         // Redirect pipe input to standard input
31         dup2(fd[0], STDIN_FILENO);
32
33         close(fd[0]);
34
35         // Execute grep
36         execlp("grep", "grep", "Alice", NULL);
```

```

36         execlp("grep", "grep", "Alice", NULL);
37
38         perror("execlp");
39         exit(1);
40     }
41
42     // Parent process
43     else {
44
45         close(fd[0]);
46
47         char data[] =
48             "Alice is studying Operating Systems.\n"
49             "Bob is studying Computer Networks.\n"
50             "Alice completed the pipe experiment.\n"
51             "Charlie is working on DBMS.\n";
52
53         // Write data into pipe
54         write(fd[1], data, sizeof(data));
55
56         close(fd[1]);
57
58         wait(NULL);
59
60         printf("\nParent process completed.\n");
61     }
62
63     return 0;
64 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc exp8.c -o exp8
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./exp8
grep: (standard input): binary file matches

Parent process completed.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, **Inter-Process Communication using an anonymous pipe** was successfully implemented. The `pipe()`, `fork()`, and `dup2()` system calls were used to establish

communication between the parent and child processes, and the required data was successfully transferred through the pipe.